

# claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-adf6feec828d4df54.jsonl`  
- SHA-256: `245b44348d5ac77be8027f3819120d5d270c7114bcf9058236c1d4dae670af3b`  
- Source modified: `2026-06-10T06:14:26+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_2a64bc88-2a8`  
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

## Transcript

### 1. user (2026-06-10T06:11:20.612Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED\_MODEL\_IMPLEMENTATION\_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

ROLE: adversarial verifier. An auditor ("eng-eval") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE\_MAP.md gotchas or DEFERRED\_HARDENING\_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.

FINDING: {"title":"Gen-0 TAL harness (Phase-0 M0.4) gaps: no score-aware metrics, partial-label windowing not integrated, no lineage enforcement","severity":"high","area":"Phase-0 readiness","file":"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/metrics.py:136-144, harness.py:47-51, regression.py, eval\_partial\_labels.py","evidence":"(a) The entire scoring spine is set-based: Interval=(start,end) with no confidence; harness.\_rows\_to\_intervals (harness.py:47-51) drops the DB confidence column; metrics.pr\_curve (136-144) is misnamed ? it is an IoU-threshold sweep, not a precision-recall curve (no ranking by score), and no AP@tIoU or mAP exists. ActionFormer/ASFormer (the named M0.4 models) emit confidence-scored proposals; standard TAL evaluation is mAP averaged over tIoU 0.3?0.7. (b) The partial-label window restriction (the methodology the human corpus requires) lives ONLY in the standalone eval\_partial\_labels.py:47-54 ? harness.evaluate, batch.aggregate, and regression.regress all score unrestricted, so running them over the prefix-labeled human corpus counts unlabeled-tail detections as FPs (the planned labeled\_window manifest field has no consumer in the core paths). (c) No GT loader or manifest carries a source/lineage field, and nothing programmatic prevents Gemini-silver CSVs from feeding training (distill\_local trains on Gemini labels today ? the plan's M0.3 itself calls this 'a live no-paid-LLM-training violation'); the only lineage mechanism is automated\_provider.warn\_if\_same\_lineage, which is warn-only and unused by the training path. (d) Manifest fps/frame\_width are hand-typed JSON numbers (output/human\_lovo\_manifest.json pattern) with no probe/cross-check ? a typo'd fps silently rescales features (x2 class error already

possible given the 30/59.94 mix).", "why\_it\_matters": "M0.4 says 'productionize rally\_seq\_proto.py reusing metrics.match\_intervals' ? but match\_intervals alone cannot evaluate a scored-proposal TAL model at parity with the literature, the corpus the harness must train/eval on is partial-labeled, and the compliance guardrail the plan calls a hard blocker has no code-level enforcement. These are the three concrete build items before Gen-0 numbers can be trusted: score-aware AP@tIoU in metrics.py, labeled\_window plumbed through harness/batch/regression, and a source field rejected-at-load for training views."}

Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

## 2. user (2026-06-10T06:11:31.469Z)

```
1      """Pure temporal interval-matching metrics for rally evaluation.
2
3      No I/O, no DB, no global state ? every function here is deterministic and unit
4      tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5      start < end. We score a list of *predicted* rally intervals against a list of
6      *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7      """
8
9      from dataclasses import dataclass, field
10     from typing import List, Tuple, Optional
11
12     Interval = Tuple[float, float]
13
14
15     def interval_iou(a: Interval, b: Interval) -> float:
16         """Temporal intersection-over-union of two intervals.
17
18         Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19         overlap / union, both measured as durations on the timeline.
20         """
21         a_start, a_end = a
22         b_start, b_end = b
23         if a_end <= a_start or b_end <= b_start:
24             return 0.0
25         inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26         if inter <= 0.0:
27             return 0.0
28         union = (a_end - a_start) + (b_end - b_start) - inter
29         return inter / union if union > 0 else 0.0
30
31
32     @dataclass
33     class Match:
34         """One matched predicted?ground-truth pair."""
35         pred_index: int
36         gt_index: int
37         iou: float
38
39
40     @dataclass
41     class MatchResult:
42         """Outcome of matching predictions to ground truth at a given IoU threshold."""
43         matches: List[Match] = field(default_factory=list)
44         unmatched_pred_indices: List[int] = field(default_factory=list) # false positives
45         unmatched_gt_indices: List[int] = field(default_factory=list)   # false negatives
46 (misses)
47
48     def match_intervals(preds: List[Interval], gts: List[Interval],
49                        iou_threshold: float = 0.5) -> MatchResult:
50         """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
```

```

52     Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53     are claimed highest-IoU first, and each prediction/GT can be used once. This
54     keeps a single prediction from being credited for two ground-truth rallies
55     (and vice versa).
56     """
57     candidates = []
58     for pi, p in enumerate(preds):
59         for gi, g in enumerate(gts):
60             iou = interval_iou(p, g)
61             if iou >= iou_threshold:
62                 candidates.append((iou, pi, gi))
63     # Highest IoU first; ties broken by indices for determinism.
64     candidates.sort(key=lambda c: (-c[0], c[1], c[2]))
65
66     used_pred, used_gt = set(), set()
67     matches: List[Match] = []
68     for iou, pi, gi in candidates:
69         if pi in used_pred or gi in used_gt:
70             continue
71         used_pred.add(pi)
72         used_gt.add(gi)
73         matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))
74
75     unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
76     unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
77     return MatchResult(matches=matches,
78                        unmatched_pred_indices=unmatched_pred,
79                        unmatched_gt_indices=unmatched_gt)
80
81
82 @dataclass
83 class Score:
84     """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
85     iou_threshold: float
86     num_pred: int
87     num_gt: int
88     true_positives: int
89     false_positives: int
90     false_negatives: int
91     precision: float
92     recall: float
93     f1: float
94     mean_iou: float          # over matched pairs (0.0 if none)
95     mean_start_error: float  # mean |pred_start - gt_start| over matches, seconds
96     mean_end_error: float    # mean |pred_end - gt_end| over matches, seconds
97
98     def as_dict(self) -> dict:
99         return self.__dict__.copy()
100
101
102 def score(preds: List[Interval], gts: List[Interval],
103          iou_threshold: float = 0.5,
104          match_result: Optional[MatchResult] = None) -> Score:
105     """Compute precision/recall/F1, mean matched IoU, and boundary errors.
106
107     Pass a precomputed `match_result` to avoid re-matching (e.g. when also
108     extracting the failure lists); otherwise matching is done here.
109     """
110     mr = match_result if match_result is not None else match_intervals(preds, gts,
111                                iou_threshold)
112     tp = len(mr.matches)
113     fp = len(mr.unmatched_pred_indices)
114     fn = len(mr.unmatched_gt_indices)
115

```

```

116     precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
117     recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
118     f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
else 0.0
119
120     if mr.matches:
121         mean_iou = sum(m.iou for m in mr.matches) / tp
122         mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
mr.matches) / tp
123         mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
mr.matches) / tp
124     else:
125         mean_iou = mean_start_err = mean_end_err = 0.0
126
127     return Score(
128         iou_threshold=iou_threshold,
129         num_pred=len(preds), num_gt=len(gts),
130         true_positives=tp, false_positives=fp, false_negatives=fn,
131         precision=precision, recall=recall, f1=f1,
132         mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
133     )
134
135
136 def pr_curve(preds: List[Interval], gts: List[Interval],
137             thresholds: Optional[List[float]] = None) -> List[Score]:
138     """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
139
140     Lets you see strict-vs-lenient behaviour rather than a single operating point.
141     """
142     if thresholds is None:
143         thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
144     return [score(preds, gts, iou_threshold=t) for t in thresholds]
145

```

### 3. user (2026-06-10T06:11:32.027Z)

```

1     """Score the pipeline's detected rallies against ground-truth annotations.
2
3     Reads predictions straight from the SQLite DB (no pipeline re-run, no API cost),
4     so scoring is fast, deterministic, and free. Two prediction "stages" can be
5     scored:
6
7     - ``candidates`` ? raw detector windows (``candidate_segments`` table). What
8       the shuttle/person detector found, before AI confirmation.
9     - ``final`` ? confirmed play segments (``play_segments`` table). After the
10      AI handoff trimmed/validated boundaries.
11
12     Comparing the two tells you whether the weak link is *detection* (candidates
13     already miss rallies) or *segmentation* (candidates catch them but final is
14     wrong) ? see docs/EVALUATION.md.
15     """
16
17     import logging
18     from dataclasses import dataclass, field
19     from typing import List, Dict, Optional
20
21     from backend.infrastructure.database import Database
22     from backend.eval import metrics
23     from backend.eval.metrics import Interval, Score, MatchResult
24
25     logger = logging.getLogger(__name__)
26
27     STAGES = ("candidates", "final")
28
29

```

```

30 @dataclass
31 class StageReport:
32     stage: str
33     pred_intervals: List[Interval]
34     score: Score
35     match_result: MatchResult
36     pr_curve: List[Score] = field(default_factory=list)
37
38
39 @dataclass
40 class EvalReport:
41     video_id: str
42     iou_threshold: float
43     gt_intervals: List[Interval]
44     stages: Dict[str, StageReport] = field(default_factory=dict)
45
46
47 def _rows_to_intervals(rows: List[dict]) -> List[Interval]:
48     """Pull (start_time, end_time) out of DB segment rows, sorted by start."""
49     ivs = [(float(r["start_time"]), float(r["end_time"])) for r in rows]
50     ivs.sort(key=lambda iv: iv[0])
51     return ivs
52
53
54 def get_predictions(db: Database, video_id: str, stage: str,
55                    detector: Optional[str] = None) -> List[Interval]:
56     """Fetch predicted rally intervals for one stage from the DB."""
57     if stage == "candidates":
58         rows = db.query_candidate_segments(video_id, detector=detector)
59     elif stage == "final":
60         rows = db.query_play_segments(video_id, {})
61     else:
62         raise ValueError(f"unknown stage {stage!r}; expected one of {STAGES}")
63     return _rows_to_intervals(rows)
64
65
66 def evaluate(db: Database, video_id: str, gt_intervals: List[Interval],
67             stages: List[str], iou_threshold: float = 0.5,
68             detector: Optional[str] = None,
69             with_pr_curve: bool = False) -> EvalReport:
70     """Score the requested stages for one video against ground truth."""
71     report = EvalReport(video_id=video_id, iou_threshold=iou_threshold,
72                        gt_intervals=gt_intervals)
73     for stage in stages:
74         preds = get_predictions(db, video_id, stage, detector=detector)
75         mr = metrics.match_intervals(preds, gt_intervals, iou_threshold)
76         s = metrics.score(preds, gt_intervals, iou_threshold, match_result=mr)
77         curve = metrics.pr_curve(preds, gt_intervals) if with_pr_curve else []
78         report.stages[stage] = StageReport(stage=stage, pred_intervals=preds,
79                                           score=s, match_result=mr, pr_curve=curve)
80     return report
81
82
83 # ----- reporting
84
85 def _fmt_ts(seconds: float) -> str:
86     seconds = max(0, int(seconds))
87     h, rem = divmod(seconds, 3600)
88     m, s = divmod(rem, 60)
89     return f"{h:02d}:{m:02d}:{s:02d}"
90
91
92 def report_to_dict(report: EvalReport) -> dict:
93     """JSON-serialisable view of an EvalReport."""
94     out = {

```

```

95         "video_id": report.video_id,
96         "iou_threshold": report.iou_threshold,
97         "num_ground_truth": len(report.gt_intervals),
98         "stages": {},
99     }
100     for stage, sr in report.stages.items():
101         out["stages"][stage] = {
102             "score": sr.score.as_dict(),
103             "pr_curve": [s.as_dict() for s in sr.pr_curve],
104         }
105     return out
106
107
108 def format_report_text(report: EvalReport, show_failures: bool = False) -> str:
109     """Human-readable console report for one video."""
110     lines = [
111         "=" * 64,
112         f" Rally evaluation ? video '{report.video_id}'",
113         f" Ground-truth rallies: {len(report.gt_intervals)} | IoU threshold:
{report.iou_threshold}",
114         "=" * 64,
115         f" {'Stage':<12} {'Pred':>5} {'TP':>4} {'FP':>4} {'FN':>4} "
116         f"{'Prec':>6} {'Rec':>6} {'F1':>6} {'mIoU':>6} {'dStart':>7} {'dEnd':>7}",
117         "-" * 64,
118     ]
119     for stage in report.stages:
120         s = report.stages[stage].score
121         lines.append(
122             f" {stage:<12} {s.num_pred:>5} {s.true_positives:>4} {s.false_positives:>4}
"
123             f"{s.false_negatives:>4} {s.precision:>6.2f} {s.recall:>6.2f} {s.f1:>6.2f} "
124             f"{s.mean_iou:>6.2f} {s.mean_start_error:>6.1f}s {s.mean_end_error:>6.1f}s"
125         )
126         lines.append("=" * 64)
127
128     if show_failures:
129         for stage, sr in report.stages.items():
130             misses = [report.gt_intervals[i] for i in
sr.match_result.unmatched_gt_indices]
131             fps = [sr.pred_intervals[i] for i in sr.match_result.unmatched_pred_indices]
132             lines.append(f"\n [{stage}] missed rallies (false negatives):
{len(misses)}")
133             for iv in misses:
134                 lines.append(f"   - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
135             lines.append(f" [{stage}] spurious detections (false positives):
{len(fps)}")
136             for iv in fps:
137                 lines.append(f"   - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
138
139     return "\n".join(lines)
140

```

#### 4. user (2026-06-10T06:11:41.809Z)

```

backend\eval\__init__.py
backend\eval\annotations.py
backend\eval\classifier.py
backend\eval\metrics.py
backend\eval\training.py
backend\eval\calibration.py
backend\eval\gemini_refine.py
backend\eval\calibrate_local.py
backend\eval\calibrate_wasb.py
backend\eval\distill_local.py
backend\eval\export_wasb_segments.py

```

```
backend\eval\harness.py
backend\eval\regression.py
backend\eval\batch.py
backend\eval\eval_partial_labels.py
backend\eval\rally_seq_proto.py
```

## 5. user (2026-06-10T06:11:42.096Z)

```
backend\eval\eval_partial_labels.py
```

## 6. user (2026-06-10T06:11:48.291Z)

```
1      """Evaluate the rally pipeline against PARTIAL human golden labels.
2
3      The VLC rally-annotator produces a human-rated CSV (rally_number,start_time,end_time,
4      ending_reason,sport). While a video is only partially tagged, the labels cover a
5      PREFIX of the video ? every rally up to some point is marked, and everything after is
6      simply un-watched (not "no rally there"). Scoring naively against such labels is wrong:
7      the detector's *correct* rallies in the un-labeled tail would be counted as false
8      positives and crush precision.
9
10     This harness fixes that by restricting the evaluation to the LABELED WINDOW
11     [window_start, window_end] (window_end defaults to the last labeled rally's end) and
12     dropping predicted segments that fall outside it. Within that window the human labels
13     ARE complete, so precision/recall/F1 are honest.
14
15     It reuses everything from the existing eval stack (no new windowing/metrics/gate logic):
16     - calibrate_wasb.make_predict_fn / load_golden_gts / BASELINE
17     - pipeline.detectors.rally_gate.apply_gate (net-crossing "gate A")
18     - eval.metrics.match_intervals / score
19     - export_wasb_segments.build_comparison / write_comparison_csv (per-rally detail CSV)
20
21     Run:
22     python -m backend.eval.eval_partial_labels \
23         --trajectory C:/Users/avidu/AppData/Local/Temp/adarsh_avi_traj.csv \
24         --labels "C:/Users/avidu/Projects/Annotation Setup/Collect/Adarsh and
25     Avi.rallies.csv" \
26         --fps 59.94 --frame-width 1920
27
28     """
29     from __future__ import annotations
30
31     import argparse
32     import os.path as osp
33     from typing import List, Optional, Tuple
34
35     from backend.eval.calibrate_wasb import make_predict_fn, load_golden_gts, BASELINE
36     from backend.eval.metrics import score
37     from backend.eval.export_wasb_segments import build_comparison, write_comparison_csv
38     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
39     from backend.pipeline.detectors import rally_gate
40
41     Interval = Tuple[float, float]
42     Config = Tuple[float, float, float]
43
44     # Operating points to compare. The tuned config wins on its fit-video but over-segments
45     # and does not transfer; baseline (+ net-crossing gate) is the stronger general point.
46     TUNED: Config = (0.05, 1.0, 1.0)
47
48     def restrict_to_window(intervals: List[Interval], w0: float, w1: float) ->
49     List[Interval]:
50         """Keep intervals that overlap the labeled window [w0, w1].
51
52         A predicted segment is evaluated iff it overlaps the labeled region; segments wholly
53         in the un-labeled tail (start >= w1) or before the window (end <= w0) are dropped so
```

```

52         they are neither rewarded nor penalized. GTs are assumed already inside the window.
53         """
54         return [(s, e) for (s, e) in intervals if e > w0 and s < w1]
55
56
57     def _eval_point(predict_fn, cfg: Config, points, fps: float, gts: List[Interval],
58                     window: Tuple[float, float], gate_crossings: int, iou: float) -> dict:
59         """Score one (config, gate) operating point inside the labeled window."""
60         preds = predict_fn(cfg)
61         if gate_crossings > 0 and points:
62             preds = rally_gate.apply_gate(points, preds, fps, min_crossings=gate_crossings)
63             preds = restrict_to_window(preds, *window)
64             sc = score(preds, gts, iou)
65             return {"cfg": cfg, "gate": gate_crossings, "n_pred": len(preds), "score": sc,
66 "preds": preds}
67
68     def run(trajjectory_csv: str, labels_csv: str, fps: float, frame_width: float,
69            window_start: float = 0.0, window_end: Optional[float] = None,
70            gate_crossings: int = 2, iou: float = 0.5,
71            comparison_out: Optional[str] = None) -> dict:
72         gts_all = load_golden_gts(labels_csv)
73         if not gts_all:
74             raise SystemExit(f"no usable human rallies in {labels_csv}")
75         if window_end is None:
76             window_end = max(e for _, e in gts_all)
77         window = (window_start, window_end)
78         gts = restrict_to_window(gts_all, *window)
79
80         predict_fn = make_predict_fn(trajjectory_csv, fps, frame_width)
81         points = TrackNetRunner.parse_trajectory_csv(trajjectory_csv)
82         traj_end = (max(p.frame for p in points) / fps) if points else 0.0
83
84         print("=== Pipeline vs PARTIAL human labels ===")
85         print(f"labels: {labels_csv}")
86         print(f" {len(gts_all)} human rallies; labeled window [{window_start:.1f}s,
87 {window_end:.1f}s] "
88             f"-> {len(gts)} rallies in-window")
89         print(f"trajectory covers ~0-{traj_end:.1f}s "
90             f"({'covers the window' if traj_end >= window_end - 1 else 'WARNING: shorter
91 than window'})")
92         print(f"IoU>={iou}; net-crossing gate K={gate_crossings}\n")
93
94         # Compare the standard operating points: baseline / baseline+gate / tuned /
95         tuned+gate.
96         points_for_gate = points
97         candidates = [
98             ("baseline", BASELINE, 0),
99             (f"baseline+gate{gate_crossings}", BASELINE, gate_crossings),
100             ("tuned", TUNED, 0),
101             (f"tuned+gate{gate_crossings}", TUNED, gate_crossings),
102         ]
103         results = []
104         header = f"{'operating point':<18} {'pred':>4} {'TP':>3} {'FN':>3} {'FP':>3} " \
105             f"{'prec':>6} {'recall':>6} {'F1':>6} {'mIoU':>6} {'sErr':>6} {'eErr':>6}"
106         print(header)
107         print("-" * len(header))
108         for name, cfg, gate in candidates:
109             r = _eval_point(predict_fn, cfg, points_for_gate, fps, gts, window, gate, iou)
110             sc = r["score"]
111             r["name"] = name
112             results.append(r)
113             print(f"name:<18} {r['n_pred']:>4} {sc.true_positives:>3}
114 {sc.false_negatives:>3} "
115                 f"{sc.false_positives:>3} {sc.precision:>6.3f} {sc.recall:>6.3f}")

```



```

{sc.f1:>6.3f} "
112         f"{sc.mean_iou:>6.3f} {sc.mean_start_error:>6.2f}
{sc.mean_end_error:>6.2f}")
113
114     best = max(results, key=lambda r: r["score"].f1)
115     print(f"\nbest operating point by F1: {best['name']} (F1 {best['score'].f1:.3f})")
116
117     # Per-rally detail for the best point: which human rallies matched/missed + extra
fires.
118     comp = build_comparison(best["preds"], gts, iou)
119     comp_out = comparison_out or osp.join("output", "adarsh_avi_vs_human.csv")
120     write_comparison_csv(comp_out, comp)
121     print(f"per-rally detail ({best['name']}) -> {comp_out}")
122
123     return {
124         "window": window, "n_human_in_window": len(gts),
125         "results": [{ "name": r["name"], "cfg": r["cfg"], "gate": r["gate"],
126                     "n_pred": r["n_pred"], **r["score"].as_dict() } for r in results],
127         "best": best["name"], "comparison_csv": comp_out,
128     }
129
130
131     # Grid centered on the two levers the human-GT failure analysis flagged:
132     # min_window_duration DOWN (recover short 3-5s rallies) and merge_gap UP (heal
133     # fragmented long rallies). Gate is swept on/off because it can over-prune.
134     DEFAULT_GRID = [
135         (vt, mg, mwd)
136         for vt in (0.01, 0.02, 0.03, 0.05)
137         for mg in (1.0, 2.0, 3.0, 4.0)
138         for mwd in (0.5, 1.0, 1.5, 2.0)
139     ]
140
141
142     def sweep(trjectory_csv: str, labels_csv: str, fps: float, frame_width: float,
143             window_start: float = 0.0, window_end: Optional[float] = None,
144             gate_options: Tuple[int, ...] = (0, 2), iou: float = 0.5,
145             grid: Optional[List[Config]] = None, top: int = 12) -> dict:
146         """Grid-search (velocity, merge_gap, min_dur) x gate vs human labels, in-window.
147
148         WARNING: this is fit-on-self on ONE video -> optimistic. It shows the achievable
149         ceiling + which levers matter, NOT a transferable config. The honest operating
150         point comes from leave-one-video-out once >=3 videos are labeled.
151         """
152         grid = grid or DEFAULT_GRID
153         gts_all = load_golden_gts(labels_csv)
154         if not gts_all:
155             raise SystemExit(f"no usable human rallies in {labels_csv}")
156         if window_end is None:
157             window_end = max(e for _, e in gts_all)
158         window = (window_start, window_end)
159         gts = restrict_to_window(gts_all, *window)
160         predict_fn = make_predict_fn(trjectory_csv, fps, frame_width)
161         points = TrackNetRunner.parse_trajectory_csv(trjectory_csv)
162
163         rows: List[dict] = []
164         for cfg in grid:
165             for gate in gate_options:
166                 r = _eval_point(predict_fn, cfg, points, fps, gts, window, gate, iou)
167                 sc = r["score"]
168                 rows.append({ "cfg": cfg, "gate": gate, "n_pred": r["n_pred"], "f1": sc.f1,
169                             "precision": sc.precision, "recall": sc.recall, "mean_iou":
sc.mean_iou,
170                             "tp": sc.true_positives, "fp": sc.false_positives, "fn":
sc.false_negatives})
171         rows.sort(key=lambda x: (x["f1"], x["mean_iou"]), reverse=True)

```

```

172
173     print(f"=== Config sweep vs human labels ({len(gts)} rallies in "
174           f"[{window_start:.0f},{window_end:.0f}]s; {len(grid)}x{len(gate_options)}
configs) ===")
175     print("NOTE: fit-on-self on ONE video = optimistic; real test is leave-one-video-
out.\n")
176     hdr = f"{'#':>3} {'veloc':>6} {'merge':>6} {'minDur':>6} {'gate':>4} " \
177           f"{'F1':>6} {'prec':>6} {'recall':>6} {'mIoU':>6} {'TP':>3} {'FP':>3}
{'FN':>3}"
178     print(hdr); print("-" * len(hdr))
179     for i, r in enumerate(rows[:top], 1):
180         vt, mg, mwd = r["cfg"]
181         print(f"{i:>3} {vt:>6.3f} {mg:>6.1f} {mwd:>6.1f} {r['gate']:>4} "
182               f"{r['f1']:>6.3f} {r['precision']:>6.3f} {r['recall']:>6.3f}
{r['mean_iou']:>6.3f} "
183               f"{r['tp']:>3} {r['fp']:>3} {r['fn']:>3}")
184     b = rows[0]
185     print(f"\nbest: velocity={b['cfg'][0]} merge_gap={b['cfg'][1]} min_dur={b['cfg'][2]}
"
186           f"gate={b['gate']} -> F1 {b['f1']:.3f} (P {b['precision']:.3f} / R
{b['recall']:.3f})")
187     return {"window": window, "n_gt": len(gts), "ranked": rows, "best": b}
188
189
190 def main() -> None:
191     ap = argparse.ArgumentParser(description=__doc__)
192     ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
193     ap.add_argument("--labels", required=True, help="human golden CSV
(rally_number,start_time,end_time,...)")
194     ap.add_argument("--fps", type=float, default=59.94)
195     ap.add_argument("--frame-width", type=float, default=1920.0)
196     ap.add_argument("--window-start", type=float, default=0.0)
197     ap.add_argument("--window-end", type=float, default=None,
198                     help="end of the labeled region (default: last labeled rally end)")
199     ap.add_argument("--gate", type=int, default=2, help="net-crossing gate K (0=off, 2
recommended)")
200     ap.add_argument("--iou", type=float, default=0.5)
201     ap.add_argument("--comparison-out", default=None)
202     ap.add_argument("--sweep", action="store_true",
203                     help="grid-search configs vs labels (fit-on-self; finds the ceiling
+ best levers)")
204     ap.add_argument("--top", type=int, default=12, help="how many sweep rows to print")
205     args = ap.parse_args()
206     if args.sweep:
207         sweep(args.trajectory, args.labels, args.fps, args.frame_width,
208               window_start=args.window_start, window_end=args.window_end,
209               iou=args.iou, top=args.top)
210     else:
211         run(args.trajectory, args.labels, args.fps, args.frame_width,
212               window_start=args.window_start, window_end=args.window_end,
213               gate_crossings=args.gate, iou=args.iou, comparison_out=args.comparison_out)
214
215
216 if __name__ == "__main__":
217     main()
218

```

## 7. user (2026-06-10T06:11:48.416Z)

```

1     """Regression runner ? verify the production tool against golden-set ground truth.
2
3     Implements the explicit ask: "given a video, get a golden set created and verify
4     its precision and recall against the production tool."* This is deliberately a thin
5     wrapper over machinery that already exists:

```

```

6
7     ground truth ? AnnotationProvider (file / vendor / oracle) [backend.annotations]
8     predictions  ? the production pipeline's stored DB rows      [backend.eval.harness]
9     comparison   ? interval matching + P/R/F1                    [backend.eval.metrics]
10    baseline     ? a per-video snapshot of "known good" numbers [this module]
11
12    A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13    baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14    later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15    independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16    ""
17    import os
18    import json
19    import logging
20    from dataclasses import dataclass, asdict
21    from typing import Dict, List, Optional
22
23    from backend.infrastructure.database import Database
24    from backend.eval import metrics
25    from backend.eval.harness import get_predictions
26    from backend.annotations import annotation_registry
27
28    logger = logging.getLogger(__name__)
29
30    DEFAULT_BASELINE_PATH = os.path.join("output", "regression_baseline.json")
31    DEFAULT_TOLERANCE = 0.05
32
33
34    @dataclass
35    class RegressionResult:
36        video_id: str
37        stage: str
38        iou_threshold: float
39        precision: float
40        recall: float
41        f1: float
42        # Baseline values compared against (None when no baseline existed yet).
43        baseline_precision: Optional[float]
44        baseline_recall: Optional[float]
45        tolerance: float
46        regressed: bool
47        reasons: List[str]
48
49        def as_dict(self) -> dict:
50            return asdict(self)
51
52
53    # ----- baseline store
54
55    def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
56        """Load the per-video baseline snapshot file (returns {} if absent)."""
57        if not os.path.isfile(path):
58            return {}
59        with open(path) as f:
60            return json.load(f)
61
62
63    def _baseline_key(video_id: str, stage: str) -> str:
64        return f"{video_id}::{stage}"
65
66
67    def save_baseline(video_id: str, stage: str, precision: float, recall: float,
68                     f1: float, path: str = DEFAULT_BASELINE_PATH) -> None:
69        """Snapshot a video/stage's current numbers as the new 'known good' baseline.
70

```

```

71 Use this to (re)baseline after a *legitimate* production improvement, so old
72 baselines don't raise false regressions. Stored as plain JSON for easy diffing.
73 """
74 baselines = load_baselines(path)
75 baselines[_baseline_key(video_id, stage)] = {
76     "video_id": video_id, "stage": stage,
77     "precision": precision, "recall": recall, "f1": f1,
78 }
79 os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
80 with open(path, "w") as f:
81     json.dump(baselines, f, indent=2, sort_keys=True)
82     logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
83 _baseline_key(video_id, stage), precision, recall)
84
85 # ----- the runner
86
87 def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
88     stage: str = "final", iou_threshold: float = 0.5,
89     detector: Optional[str] = None,
90     tolerance: float = DEFAULT_TOLERANCE,
91     baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
92     """Score stored predictions for one video against ground truth and compare to
93 baseline.
94
95     `ground_truth` is a list of (start, end) intervals ? typically obtained from an
96     AnnotationProvider (see `regress_with_provider`). A regression is flagged when
97     precision OR recall falls more than `tolerance` below the snapshotted baseline.
98     If no baseline exists yet, `regressed` is False (nothing to compare to) and the
99     caller can choose to snapshot the current numbers via `save_baseline`.
100
101     preds = get_predictions(db, video_id, stage, detector=detector)
102     s = metrics.score(preds, ground_truth, iou_threshold)
103
104     baselines = load_baselines(baseline_path)
105     b = baselines.get(_baseline_key(video_id, stage))
106
107     reasons: List[str] = []
108     regressed = False
109     base_p = base_r = None
110     if b is not None:
111         base_p, base_r = b["precision"], b["recall"]
112         if s.precision < base_p - tolerance:
113             regressed = True
114             reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
115 {tolerance}")
116         if s.recall < base_r - tolerance:
117             regressed = True
118             reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
119 {tolerance}")
120         else:
121             reasons.append("no baseline recorded for this video/stage ? nothing to compare")
122
123     return RegressionResult(
124         video_id=video_id, stage=stage, iou_threshold=iou_threshold,
125         precision=s.precision, recall=s.recall, f1=s.f1,
126         baseline_precision=base_p, baseline_recall=base_r,
127         tolerance=tolerance, regressed=regressed, reasons=reasons,
128     )
129
130 def regress_with_provider(db: Database, video_id: str, video_ref: str,
131     tier: str = "file", stage: str = "final",
132     iou_threshold: float = 0.5,
133     detector: Optional[str] = None,

```

```

132         tolerance: float = DEFAULT_TOLERANCE,
133         baseline_path: str = DEFAULT_BASELINE_PATH,
134         guideline: Optional[str] = None,
135         sport: str = "badminton",
136         provider_kwargs: Optional[dict] = None) -> RegressionResult:
137     """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
138
139     `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
140     once those providers land). `video_ref` is whatever that provider expects (a CSV
141     path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
142     predictions in the DB (the file MD5).
143     """
144     provider = annotation_registry.get(tier, **(provider_kwargs or {}))
145     gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
146     return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
147                  detector=detector, tolerance=tolerance, baseline_path=baseline_path)
148
149
150     def format_result(r: RegressionResult) -> str:
151         verdict = "REGRESSED" if r.regressed else "ok"
152         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
153               f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
154         if r.baseline_precision is not None:
155             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
156         return head + ".join(f"\n    - {why}" for why in r.reasons)
157

```

## 8. assistant (2026-06-10T06:11:59.726Z)

Now the plan doc and the remaining cited files.

## 9. user (2026-06-10T06:12:01.759Z)

```

89- ~40-min GPU jobs, and it would fracture the one codebase holding the QA-state + eval
harness + provider registry.
90-- **On-device (deferred) = the only place a 2nd language appears, and it doesn't dictate our
dev language:** train in
91- Python ? **export across the boundary** (ONNX/Core ML/ExecuTorch/GGUF) ? a thin device
shell (Swift/Kotlin/C++/Rust)
92- runs the artifact with zero Python. **The one concrete constraint now: train with export-
clean ops** so the compression
93: target stays reachable without re-architecting. (On-device base stays on the Qwen3-VL-4B
Apache lineage ? *not* Gemma-3-270M.)
94-- Escalation order if a real Python CPU hot path ever appears: free-threading ?
numpy/vectorize ? Cython/PyO3 ? only then a
95- separate service. **None exists today.**
96-
97---
--
101-### M0.1 ? The labeled-corpus spine **+ the event-index contract** (the moat) . *greenlight
item*
102-Canonical JSONL row per rally (corpus) **and** the queryable per-video event-index schema
(the conversational seam)
103-? same row, wired to a store. Fields: `{video_id(MD5), rally_ordinal, sport, start, end,
ending_reason/fault_tags,
104-shot_count, label_type, source(human|pseudo|vendor), consent/training_opt_in, split(BY
MATCH), trajectory_ref,
105:labeled_window, derived_clip_ref, confidence}`. 3 transcoders (heuristic/LogReg · TAL
ActivityNet-JSON · VLM clip?QA),
106:one scorer spine (`metrics.match_intervals`). Authored in `sports-data-collector` (system-
of-record; extends PR #13).
107-**Reserve fault\event taxonomy now.** Acceptance: round-trip test; split-by-match enforced;
**no Gemini-sourced rows in any training view**.

```

```

108-
109-### M0.2 ? Grow the corpus . **DONE (n=6), running** . *owner action + my scoring loop*
110-6 distinct matches labeled (3 singles incl. Mahadevpura, 1 doubles, 2 multi-court); per-
match + LOVO recorded; the
111-"contrast" data-quality metric validated. Owner adding 1 more varied multi-court-club match
? margin. **Bottleneck =
112-data + calibration, not method.** Keep growing per new sport.
113-
114-### M0.3 ? Compliance cleanup (**hard blocker ? do before any training run**) . *greenlight
item*
115-- *(i) PURGE the Gemini-derived TRAINING labels** in `training.label_candidates` (over
Gemini CSVs) ? a **live
116- no-paid-LLM-training violation** that bakes in the moment a training run precedes it.
Retarget to human + open-teacher
117- (own-model/WASB) labels; Gemini = eval/reference/fallback only.
118-- *(ii) WASB C3 ? first-class action item (owner):** the academic-data checkpoint
provenance is a **commercial-ship
119- blocker** that a clean head does NOT launder. Resolve via **own-trained weights** or a
clean detector swap; **multiplies
120- per sport.** Carry as a tracked blocker that **gates SHIP** (not Gen-0 research).
121-
122-### M0.4 ? Gen-0 TAL harness (**FIRST-CLASS ACTION ITEM** per owner) . *greenlight item*
123-Productionize `rally_seq_proto.py` ? a **one-command train?eval?ship-gate** harness over
**ActionFormer (MIT, 1:1
124-rally=instance)** and/or **ASFormer (MIT, TAS)** (use `sj-li/MS-TCN2` if MS-TCN++, *not* the
Commons-Clause repo),
125:reusing `metrics.match_intervals`. **Define the ship-gate target up front** (open item): a
FLOOR of "beat the honest n=6
126-heuristic LOVO **0.480**" is necessary but not sufficient ? set an explicit **F1@tIoU**
target for "very high P/R" (e.g.
127-F1@tIoU=0.5 for highlights, tighter for precise rally cuts) before declaring victory. Trains
on the GPU/WSL box.
128-
129-
130-
131:## Phase 1 ? Gen-0 ship decision (gated on n?5 + M0.4)
132-Run the A/B; if the learned Gen-0 clears the ship-gate with per-video calibration, it
becomes the rally/highlight
133-extractor (local, MIT). If not, the gap (more data/features) is the next signal ? no
overfitting to declare victory.
134-
135-## Phase 2 ? Gen-1 / Gen-2 (gated on platform scaffolding P0?P4 + healthy corpus + explicit
go)
--
146-
147-## Decisions / status
148-| # | item | status |
149-|---|---|---|
150-| 1 | Greenlight Phase-0 milestone(s) ? rec: **M0.1 corpus+event-index**, then **M0.4
harness** | awaiting owner |
151-| 2 | WASB C3 ? first-class action item (own-weights \ clean swap) | **owner: yes** ? |
152-| 3 | Ratify licensing **MIT \ Apache-2.0 \ BSD** (code+data+weights) | **owner: yes** ? |
153-| 4 | Collector = corpus + event-store system-of-record | **owner: confirm?** |
154-| 5 | Rename repo off "badminton" | **owner action** ? |
--
161-training . on-device 4B latency on 8 GB unmeasured.
162-
163-
164-*Design-only and reversible. Nothing starts until you greenlight a milestone. Sequencing
discipline holds: the committed
165:next platform slice remains the async job model (single-tenant); only the M0.1?M0.4
foundation is safe to start.*

```

```

1      """Batch evaluation across a collector export manifest.
2
3      Phase 3 of the collector->indexer bridge: walk the `manifest.json` emitted by
4      sports-data-collector's `curate export`, (optionally) process each video through
5      a segmenter so detections land in the indexer DB, then score every video's
6      detected rallies against its ground-truth CSV and aggregate the results.
7
8      This module holds the *pure* pieces (path resolution, stage selection, metric
9      aggregation) so they can be unit-tested without touching video files, the GPU,
10     or any API. The orchestration/side-effects live in `run_phase3_eval.py`.
11     """
12
13     import os
14     from typing import Dict, List, Optional
15
16     # Segmenters that only ever populate the `final` play_segments table (no raw
17     # candidate stage), so scoring `candidates` for them would be meaninglessly empty.
18     _FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}
19
20
21
22     def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
23         """Resolve a manifest `local_path` to an absolute path.
24
25         The collector stores paths like ``./data/videos\\shuttleaset_1.mp4`` relative
26         to its own repo root and with Windows separators. Normalise separators, strip
27         a leading ``./``, and resolve relative paths against `videos_root` (the
28         collector root). Returns None for a missing/empty path.
29         """
30         if not local_path:
31             return None
32         p = local_path.replace("\\", "/")
33         if p.startswith("./"):
34             p = p[2:]
35         # Cross-platform: treat Windows drive-letter absolute paths (C:/...) as absolute
36         # even when running on Linux (os.path.isabs("C:/...") == False on Unix).
37         # This keeps collector manifest paths working in tests and mixed environments.
38         if (len(p) > 1 and p[1] == ":" and p[0].isalpha()) or os.path.isabs(p):
39             return os.path.normpath(p)
40         return os.path.normpath(os.path.join(videos_root, p))
41
42
43     def default_stages_for(segmenter: str, requested: str = "auto") -> List[str]:
44         """Pick which prediction stages to score.
45
46         ``auto`` scores both stages for two-stage detectors (yolo_hybrid/tracknet_*)
47         but only ``final`` for segmenters that don't emit candidates. An explicit
48         request ('candidates' | 'final' | 'both') overrides the heuristic.
49         """
50         if requested == "both":
51             return ["candidates", "final"]
52         if requested in ("candidates", "final"):
53             return [requested]
54         # auto
55         if segmenter in _FINAL_ONLY_SEGMENTERS:
56             return ["final"]
57         return ["candidates", "final"]
58
59
60     def _safe_div(num: float, den: float) -> float:
61         return num / den if den else 0.0
62
63
64     def aggregate(per_video: List[dict], stages: List[str]) -> Dict[str, dict]:
65         """Aggregate per-video stage scores into corpus-level metrics.

```

```

66
67 `per_video` is a list of dicts shaped like ``report_to_dict`` output (each has
68 ``stages[stage]["score"]`` with tp/fp/fn/mean_iou/...). For each stage we
69 compute **micro** precision/recall/F1 (pooling TP/FP/FN across all videos) and
70 a TP-weighted mean IoU, plus the macro mean of per-video F1.
71 """
72 out: Dict[str, dict] = {}
73 for stage in stages:
74     tp = fp = fn = 0
75     iou_weighted = 0.0
76     f1s: List[float] = []
77     n_videos = 0
78     for rep in per_video:
79         sr = rep.get("stages", {}).get(stage)
80         if not sr:
81             continue
82         s = sr["score"]
83         tp += s["true_positives"]
84         fp += s["false_positives"]
85         fn += s["false_negatives"]
86         iou_weighted += s["mean_iou"] * s["true_positives"]
87         f1s.append(s["f1"])
88         n_videos += 1
89     precision = _safe_div(tp, tp + fp)
90     recall = _safe_div(tp, tp + fn)
91     out[stage] = {
92         "videos": n_videos,
93         "tp": tp, "fp": fp, "fn": fn,
94         "micro_precision": precision,
95         "micro_recall": recall,
96         "micro_f1": _safe_div(2 * precision * recall, precision + recall),
97         "macro_f1": _safe_div(sum(f1s), len(f1s)),
98         "mean_iou": _safe_div(iou_weighted, tp),
99     }
100 return out
101
102
103 def format_aggregate(agg: Dict[str, dict]) -> str:
104     """Human-readable corpus-level summary table."""
105     lines = [
106         "=" * 78,
107         " CORPUS AGGREGATE",
108         "=" * 78,
109         f" {'Stage':<12} {'Vids':>4} {'TP':>5} {'FP':>5} {'FN':>5} "
110         f"{'mPrec':>6} {'mRec':>6} {'microF1':>8} {'macroF1':>8} {'mIoU':>6}",
111         "-" * 78,
112     ]
113     for stage, a in agg.items():
114         lines.append(
115             f" {stage:<12} {a['videos']:>4} {a['tp']:>5} {a['fp']:>5} {a['fn']:>5} "
116             f"{a['micro_precision']:>6.2f} {a['micro_recall']:>6.2f} "
117             f"{a['micro_f1']:>8.2f} {a['macro_f1']:>8.2f} {a['mean_iou']:>6.2f}"
118         )
119     lines.append("=" * 78)
120     return "\n".join(lines)
121

```

## 11. user (2026-06-10T06:12:27.659Z)

```

1     """PROTOTYPE: a learned per-FRAME rally segmenter (vs the unsupervised windowing
2 heuristic).
3
4 Motivation (2026-06-08 finding): WASB tracks the shuttle well on Kushagra (97% of GT
5 rallies contain >=2 net-crossings) yet NO velocity-windowing config segments it
(per-video F1 ceiling 0.163), while it works on Adarsh (0.73). The rally SIGNAL is in

```



```

6     the trajectory; the hand-tuned heuristic just can't extract it across footage. This
7     prototype tests the hypothesis that a LEARNED model on per-frame trajectory features
8     recovers the rally segments ? including on the footage the heuristic fails on.
9
10    Approach (deliberately small, dependency-light: numpy + scipy + the repo's pure-python
11    LogisticRegression ? no torch/sklearn):
12        1. Per-frame features over a local window from the WASB trajectory: visibility rate,
13           mean/max shuttle speed (frame-width-normalized), net-crossing RATE, x/y spread.
14           All are scale/camera-robust RATES (unlike a raw velocity threshold).
15        2. Per-frame label = inside a human GT rally or not.
16        3. LEAVE-ONE-VIDEO-OUT: train the classifier on the other match(es), test on the
17           held-out one (the honest cross-footage number; no within-video leakage).
18        4. Smooth probabilities -> threshold (tuned on TRAIN) -> contiguous in-rally segments
19           -> merge tiny gaps / drop short -> score vs GT at IoU>=0.5 (same metric as the
    heuristic).
20
21    This is a directional PROOF-OF-SIGNAL, not a production model (n=2 videos, linear model,
22    hand features). It exists to answer one question: is the rally boundary learnable from
23    the trajectory where the heuristic fails? Compare its held-out F1 to the heuristic LOVO.
24
25    Run:
26        python -m backend.eval.rally_seq_proto --manifest output/human_lovo_manifest.json
27    """
28    from __future__ import annotations
29
30    import json
31    import argparse
32    from typing import Dict, List, Tuple
33
34    import numpy as np
35    from scipy.ndimage import uniform_filter1d, maximum_filter1d
36
37    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
38    from backend.eval.classifier import LogisticRegression
39    from backend.eval.calibrate_wasb import load_golden_gts
40    from backend.eval.metrics import score
41
42    FEAT_NAMES = ["vis_rate", "spd_mean", "spd_max", "cross_rate", "x_std", "y_std"]
43    Interval = Tuple[float, float]
44
45
46    def _spread(a: np.ndarray) -> float:
47        return float(np.percentile(a, 95) - np.percentile(a, 5)) if a.size > 1 else 0.0
48
49
50    def build_frame_arrays(points, frame_width: float) -> Dict:
51        """Per-frame trajectory arrays + per-visible-frame speed and net-crossing events."""
52        pts = sorted(points, key=lambda p: p.frame)
53        N = (pts[-1].frame + 1) if pts else 1
54        vis = np.zeros(N); x = np.zeros(N); y = np.zeros(N)
55        for p in pts:
56            if 0 <= p.frame < N and p.visible and p.x >= 0:
57                vis[p.frame] = 1.0; x[p.frame] = float(p.x); y[p.frame] = float(p.y)
58
59        mask = vis > 0
60        play_is_x = _spread(x[mask]) >= _spread(y[mask])
61        coord = x if play_is_x else y
62        net = float(np.median(coord[mask])) if mask.any() else 0.0
63        deadband = 0.06 * (_spread(coord[mask]) if mask.sum() > 1 else 1.0)
64
65        speed = np.zeros(N); spd_mask = np.zeros(N); cross = np.zeros(N)
66        prev = None
67        for p in pts:
68            f = p.frame
69            if not (0 <= f < N and vis[f]):

```

```

70         continue
71     if prev is not None:
72         df = f - prev
73         if df > 0:
74             speed[f] = float(np.hypot(x[f] - x[prev], y[f] - y[prev]) / frame_width
/ df)
75             spd_mask[f] = 1.0
76             c, pc = coord[f] - net, coord[prev] - net
77             if abs(c) > deadband and abs(pc) > deadband and (c > 0) != (pc > 0):
78                 cross[f] = 1.0
79         prev = f
80     return dict(N=N, vis=vis, x=x, y=y, speed=speed, spd_mask=spd_mask, cross=cross,
fw=frame_width)
81
82
83     def features(arr: Dict, fps: float, win_sec: float = 0.75, stride_sec: float = 0.1):
84         """Windowed per-frame features sampled at a stride -> (X[n_samples,6],
times[n_samples])."""
85         N = arr["N"]; fw = arr["fw"]
86         size = max(3, int(win_sec * fps) * 2 + 1)
87         vis, speed, spd_mask, cross, x, y = (arr[k] for k in ("vis", "speed", "spd_mask",
"cross", "x", "y"))
88
89         n = uniform_filter1d(vis, size, mode="constant") # mean
visibility (== vis_rate)
90         sm = uniform_filter1d(spd_mask, size, mode="constant")
91         spd_mean = uniform_filter1d(speed, size, mode="constant") / np.maximum(sm, 1e-9)
92         spd_max = maximum_filter1d(speed, size, mode="constant")
93         cross_rate = uniform_filter1d(cross, size, mode="constant") * fps # crossings
/ second
94         xm = uniform_filter1d(x * vis, size, mode="constant") / np.maximum(n, 1e-9)
95         x2 = uniform_filter1d((x * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
96         x_std = np.sqrt(np.maximum(x2 - xm ** 2, 0.0)) / fw
97         ym = uniform_filter1d(y * vis, size, mode="constant") / np.maximum(n, 1e-9)
98         y2 = uniform_filter1d((y * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
99         y_std = np.sqrt(np.maximum(y2 - ym ** 2, 0.0)) / fw
100
101         idx = np.arange(0, N, max(1, int(stride_sec * fps)))
102         X = np.stack([n[idx], spd_mean[idx], spd_max[idx], cross_rate[idx], x_std[idx],
y_std[idx]], axis=1)
103         return np.nan_to_num(X), idx / fps
104
105
106     def labels_for(times: np.ndarray, gts: List[Interval]) -> np.ndarray:
107         y = np.zeros(len(times))
108         for i, t in enumerate(times):
109             for s, e in gts:
110                 if s <= t <= e:
111                     y[i] = 1.0
112                     break
113         return y
114
115
116     def roc_auc(y: np.ndarray, p: np.ndarray) -> float:
117         """Rank-based AUC (Mann-Whitney); pure numpy."""
118         pos, neg = p[y > 0.5], p[y <= 0.5]
119         if pos.size == 0 or neg.size == 0:
120             return float("nan")
121         order = np.argsort(p, kind="mergesort")
122         ranks = np.empty(len(p)); ranks[order] = np.arange(1, len(p) + 1)
123         return float((ranks[y > 0.5].sum() - pos.size * (pos.size + 1) / 2) / (pos.size *
neg.size))
124
125
126     def probs_to_segments(probs: np.ndarray, times: np.ndarray, threshold: float,

```

```

127             merge_gap: float = 1.0, min_dur: float = 1.0, smooth: int = 7) ->
List[Interval]:
128     p = uniform_filter1d(probs, max(1, smooth), mode="nearest")
129     on = p >= threshold
130     runs: List[Interval] = []
131     i, n = 0, len(on)
132     while i < n:
133         if on[i]:
134             j = i
135             while j + 1 < n and on[j + 1]:
136                 j += 1
137             runs.append((float(times[i]), float(times[j])))
138             i = j + 1
139         else:
140             i += 1
141     merged: List[Interval] = []
142     for s, e in runs:
143         if merged and s - merged[-1][1] <= merge_gap:
144             merged[-1] = (merged[-1][0], e)
145         else:
146             merged.append((s, e))
147     return [(s, e) for s, e in merged if e - s >= min_dur]
148
149
150 def _seg_f1(clf, vids: List[Dict], thr: float) -> float:
151     f1s = []
152     for v in vids:
153         segs = probs_to_segments(clf.predict_proba(v["X"]), v["times"], thr)
154         f1s.append(score(segs, v["gts"], 0.5).f1)
155     return sum(f1s) / max(len(f1s), 1)
156
157
158 def run(specs: List[Dict]) -> dict:
159     data = {}
160     for s in specs:
161         pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
162         arr = build_frame_arrays(pts, float(s["frame_width"]))
163         X, times = features(arr, float(s["fps"]))
164         gts = load_golden_gts(s["labels"])
165         data[s["name"]] = {"X": X, "times": times, "y": labels_for(times, gts), "gts":
gts}
166     names = list(data)
167     print(f"=== Learned per-frame rally segmenter ? LEAVE-ONE-VIDEO-OUT ({len(names)}
videos) ===")
168     print(f"features: {FEAT_NAMES}\n")
169
170     results = []
171     for test in names:
172         train = [data[n] for n in names if n != test]
173         Xtr = np.vstack([v["X"] for v in train]); ytr = np.concatenate([v["y"] for v in
train])
174         clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
175         clf.fit(Xtr, ytr, feature_names=FEAT_NAMES)
176         # tune the decision threshold on TRAIN segment-F1 (no test leakage)
177         thr = max(np.arange(0.2, 0.81, 0.05), key=lambda t: _seg_f1(clf, train,
float(t)))
178         v = data[test]
179         probs = clf.predict_proba(v["X"])
180         auc = roc_auc(v["y"], probs)
181         segs = probs_to_segments(probs, v["times"], float(thr))
182         sc = score(segs, v["gts"], 0.5)
183         # Oracle threshold (fit-on-TEST, upper bound): isolates the representation
ceiling
184         # from the threshold-transfer gap. Optimistic ? for diagnosis only.
185         othr = max(np.arange(0.2, 0.81, 0.05),

```

```

186             key=lambda t: score(probs_to_segments(probs, v["times"], float(t)),
v["gts"], 0.5).f1)
187         osc = score(probs_to_segments(probs, v["times"], float(othr)), v["gts"], 0.5)
188         results.append({"video": test, "thr": float(thr), "auc": auc,
189                        "f1": sc.f1, "precision": sc.precision, "recall": sc.recall,
190                        "n_pred": len(segs), "n_gt": len(v["gts"]),
191                        "oracle_thr": float(othr), "oracle_f1": osc.f1})
192         print(f"[held-out {test}] per-frame AUC {auc:.3f} | LOVO-threshold F1
{sc.f1:.3f} "
193               f"(P {sc.precision:.3f} R {sc.recall:.3f}, thr {thr:.2f} from train) | "
194               f"oracle-threshold F1 {osc.f1:.3f} (thr {othr:.2f}, fit-on-test upper
bound)")
195         mean_f1 = sum(r["f1"] for r in results) / len(results)
196         print(f"\nlearned LOVO mean held-out F1: {mean_f1:.3f}")
197         return {"per_video": results, "mean_f1": mean_f1}
198
199
200     def main() -> None:
201         ap = argparse.ArgumentParser(description=__doc__)
202         ap.add_argument("--manifest", required=True, help="JSON:
[{name,trajectory,fps,frame_width,labels}, ...]")
203         args = ap.parse_args()
204         with open(args.manifest, encoding="utf-8") as f:
205             run(json.load(f))
206
207
208     if __name__ == "__main__":
209         main()
210

```

## 12. user (2026-06-10T06:12:29.051Z)

backend\eval\calibration.py:20:List[Interval]` (e.g. WASB trajectory ? windows at those thresholds), so the same

backend\eval\calibration.py:81: windows), so scoring them against a \*fold subset\* of GT would unfairly count the

backend\eval\calibration.py:82: other rallies' windows as false positives ? precision/F1 are not well-defined

backend\eval\calibrate\_wasb.py:5: -> TrackNetRunner.trajectory\_to\_action\_windows(cfg)

[predict\_fn]

backend\eval\calibrate\_wasb.py:29:Config = Tuple[float, float, float] # (velocity\_thresh, merge\_gap, min\_window\_duration)

backend\eval\calibrate\_wasb.py:31:BASELINE: Config = (0.02, 2.0, 2.0) # the WasbConfig / trajectory\_to\_action\_windows defaults

backend\eval\calibrate\_wasb.py:53: """predict\_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""

backend\eval\calibrate\_wasb.py:58: wins = TrackNetRunner.trajectory\_to\_action\_windows(

backend\eval\calibrate\_wasb.py:60: velocity\_thresh=vt, merge\_gap=mg,

min\_window\_duration=mwd,

backend\eval\classifier.py:9:It deliberately stays simple: the rally-window features are few and roughly

backend\eval\classifier.py:42: """Fit with optional balanced class weighting (rally windows are imbalanced

backend\eval\eval\_partial\_labels.py:11:[window\_start, window\_end] (window\_end defaults to the last labeled rally's end) and

backend\eval\eval\_partial\_labels.py:12:dropping predicted segments that fall outside it. Within that window the human labels

backend\eval\eval\_partial\_labels.py:15:It reuses everything from the existing eval stack (no new windowing/metrics/gate logic):

backend\eval\eval\_partial\_labels.py:47:def restrict\_to\_window(intervals: List[Interval], w0: float, w1: float) -> List[Interval]:

backend\eval\eval\_partial\_labels.py:48: """Keep intervals that overlap the labeled window [w0, w1].

backend\eval\eval\_partial\_labels.py:51: in the un-labeled tail (start >= w1) or before the window (end <= w0) are dropped so

```

backend\eval\eval_partial_labels.py:52: they are neither rewarded nor penalized. GTs are
assumed already inside the window.
backend\eval\eval_partial_labels.py:58:         window: Tuple[float, float],
gate_crossings: int, iou: float) -> dict:
backend\eval\eval_partial_labels.py:59:     """Score one (config, gate) operating point inside
the labeled window."""
backend\eval\eval_partial_labels.py:63:     preds = restrict_to_window(preds, *window)
backend\eval\eval_partial_labels.py:69:         window_start: float = 0.0, window_end:
Optional[float] = None,
backend\eval\eval_partial_labels.py:75:     if window_end is None:
backend\eval\eval_partial_labels.py:76:         window_end = max(e for _, e in gts_all)
backend\eval\eval_partial_labels.py:77:     window = (window_start, window_end)
backend\eval\eval_partial_labels.py:78:     gts = restrict_to_window(gts_all, *window)
backend\eval\eval_partial_labels.py:86:     print(f"  {len(gts_all)} human rallies; labeled
window [{window_start:.1f}s, {window_end:.1f}s] ")
backend\eval\eval_partial_labels.py:87:         f"-> {len(gts)} rallies in-window")
backend\eval\eval_partial_labels.py:89:         f"({'covers the window' if traj_end >=
window_end - 1 else 'WARNING: shorter than window'})")
backend\eval\eval_partial_labels.py:106:         r = _eval_point(predict_fn, cfg,
points_for_gate, fps, gts, window, gate, iou)
backend\eval\eval_partial_labels.py:124:         "window": window, "n_human_in_window": len(gts),
backend\eval\eval_partial_labels.py:132:# min_window_duration DOWN (recover short 3-5s rallies)
and merge_gap UP (heal
backend\eval\eval_partial_labels.py:143:         window_start: float = 0.0, window_end:
Optional[float] = None,
backend\eval\eval_partial_labels.py:146:     """Grid-search (velocity, merge_gap, min_dur) x gate
vs human labels, in-window.
backend\eval\eval_partial_labels.py:156:     if window_end is None:
backend\eval\eval_partial_labels.py:157:         window_end = max(e for _, e in gts_all)
backend\eval\eval_partial_labels.py:158:     window = (window_start, window_end)
backend\eval\eval_partial_labels.py:159:     gts = restrict_to_window(gts_all, *window)
backend\eval\eval_partial_labels.py:166:         r = _eval_point(predict_fn, cfg, points,
fps, gts, window, gate, iou)
backend\eval\eval_partial_labels.py:174:         f"[{window_start:.0f},{window_end:.0f}]s;
{len(grid)}x{len(gate_options)} configs) ==")
backend\eval\eval_partial_labels.py:187:     return {"window": window, "n_gt": len(gts),
"ranked": rows, "best": b}
backend\eval\eval_partial_labels.py:196:     ap.add_argument("--window-start", type=float,
default=0.0)
backend\eval\eval_partial_labels.py:197:     ap.add_argument("--window-end", type=float,
default=None,
backend\eval\eval_partial_labels.py:208:         window_start=args.window_start,
window_end=args.window_end,
backend\eval\eval_partial_labels.py:212:         window_start=args.window_start,
window_end=args.window_end,
backend\eval\audio\eval_probe.py:5:WASB windows miss (our recall bottleneck), and (c) form usable
audio-only rally
backend\eval\audio\eval_probe.py:62:         f"{len(gts)} labeled rallies, {len(wasb)} WASB
baseline windows")
backend\eval\export_wasb_segments.py:8:Reuses (no new windowing/metrics/cutting logic):
backend\eval\export_wasb_segments.py:145:     # the shuttle to cross the net >= min_crossings
times within the window.
backend\eval\export_wasb_segments.py:212:         help="gate A: drop windows whose
shuttle crosses the net < N times "
backend\eval\calibrate_local.py:6:     WASB trajectory CSV ->
trajectory_to_action_windows(velocity, merge_gap, min_dur)
backend\eval\calibrate_local.py:7:         -> rally_gate (min_crossings)
-> rally windows
backend\eval\calibrate_local.py:15:Reuses: TrackNetRunner windowing, rally_gate,
calibrate_wasb.load_golden_gts,
backend\eval\calibrate_local.py:34:# (velocity_thresh, merge_gap, min_window_duration,
min_crossings)
backend\eval\calibrate_local.py:36:BASELINE_LOCAL: LocalConfig = (0.02, 2.0, 2.0, 0) # today's
local default (windowing, no gate)
backend\eval\calibrate_local.py:40:     """predict_fn(cfg) -> rally windows from a cached WASB

```

```

trajectory, LOCAL ONLY
backend\eval\calibrate_local.py:41: (windowing thresholds + the shuttle net-crossing gate).
Parsd once."""
backend\eval\calibrate_local.py:46: wins = TrackNetRunner.trajectory_to_action_windows(
backend\eval\calibrate_local.py:48: velocity_thresh=vt, merge_gap=mg,
min_window_duration=mwd,
backend\eval\distill_local.py:9: WASB trajectory -> recall-oriented candidate windows (propose
many; classifier
backend\eval\distill_local.py:12: -> rally windows.
backend\eval\distill_local.py:45: BASELINE_LOCAL = (0.02, 2.0, 2.0) # today's production-ish
windowing, no learning
backend\eval\distill_local.py:53: wins = TrackNetRunner.trajectory_to_action_windows(
backend\eval\distill_local.py:55: velocity_thresh=vt, merge_gap=mg,
min_window_duration=mwd)
backend\eval\distill_local.py:57: gated = rally_gate.gate_windows(points, intervals, fps,
min_crossings=0) # for net-crossings
backend\eval\distill_local.py:72: def baseline_windows(trajectory_csv: str, fps: float,
frame_width: float) -> List[Interval]:
backend\eval\distill_local.py:75: wins = TrackNetRunner.trajectory_to_action_windows(
backend\eval\distill_local.py:77: velocity_thresh=vt, merge_gap=mg,
min_window_duration=mwd)
backend\eval\distill_local.py:87: "y": y, "gts": gts, "baseline":
baseline_windows(spec["trajectory"], fps, fw)}
backend\eval\harness.py:7: - ``candidates`` ? raw detector windows (``candidate_segments``
table). What
backend\eval\gemini_refine.py:5: each candidate window is cut to a small padded clip and handed
to Gemini, which ?
backend\eval\gemini_refine.py:71: def refine_window(api_key: str, model: str, sport_profile,
video: str, window: Interval,
backend\eval\gemini_refine.py:77: s, e = window
backend\eval\gemini_refine.py:84: return idx, window, []
backend\eval\gemini_refine.py:98: return idx, window, mapped
backend\eval\gemini_refine.py:122: futs = [ex.submit(refine_window, api_key, model,
sport_profile, video, w, pad, duration,
backend\eval\gemini_refine.py:126: idx, window, mapped = fut.result()
backend\eval\gemini_refine.py:127: per_candidate.append((idx, window, mapped))
backend\eval\gemini_refine.py:130: print(f" cand {idx:>2}
{seconds_to_mmss(window[0])}-{seconds_to_mmss(window[1])} -> {tag}", flush=True)
backend\eval\rally_seq_proto.py:1: """PROTOTYPE: a learned per-FRAME rally segmenter (vs the
unsupervised windowing heuristic).
backend\eval\rally_seq_proto.py:4: rallies contain >=2 net-crossings) yet NO velocity-windowing
config segments it
backend\eval\rally_seq_proto.py:12: 1. Per-frame features over a local window from the WASB
trajectory: visibility rate,
backend\eval\training.py:5: produces candidate rally windows with features; the collector's
ground-truth
backend\eval\training.py:6: rally intervals tell us which windows are real. A lightweight
classifier learns
backend\eval\training.py:11: - ``label_candidates`` ? generic: label windows positive/negative
by whether they
backend\eval\training.py:26: # YOLO Stage-1 persists these per candidate window in
``candidate_segments.stats``
backend\eval\training.py:27: # (see HybridYoloSegmenter). Duration is derived from the window
bounds.
backend\eval\training.py:52: """Label each candidate window 1 (real rally) or 0 (spurious).
backend\eval\training.py:54: A window is positive iff it is matched one-to-one to a ground-
truth rally at
backend\eval\training.py:57: works for YOLO or WASB-derived windows alike.

```

### 13. user (2026-06-10T06:12:42.665Z)

```

docs\CODE_MAP.md:137: [Omitted long matching line]
docs\NEXT_STEPS.md:45: WASB-transfer trajectories: `%TEMP%\{pickleball,tabletennis}_traj.csv`.
Code: `backend/eval/{eval_partial_labels,rally_seq_proto,calibrate_local}.py`.
docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:105: labeled_window, derived_clip_ref, confidence}`. 3

```

```

transcoders (heuristic/LogReg · TAL ActivityNet-JSON · VLM clip?QA),
docs\PLATFORM_ARCHITECTURE.md:347:| **Together AI** | Managed (tune + serve) | Qwen3-VL /
Gemma-3 / Llama-4; image data | verify | partial | ? adapters downloadable | managed Qwen3-VL +
serving |
docs\PLATFORM_ARCHITECTURE.md:348:| **Fireworks AI** | Managed (tune + serve) | Full Qwen2.5-VL
(3B?72B SFT; LoRA@32B + LoRA-upload) | **7B** | partial | ?? verify export (lock-in risk) |
managed *small*-VL + serving |
docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md:92:matches. ? **The new partial-label
windowing** (`eval_partial_labels.restrict_to_window`) must flow into the
docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md:93:TAL/VLM views so prefix-labeled videos
aren't mis-scored (a corpus that's partially tagged is the norm).
docs\archives\quality-iterations\README.md:19:- ***2026-06-human-gt-tuning/`*** ? **COMPLETED
(2026-06-08).** First eval against **human** ground truth on a partially-labeled video: built
the partial-label eval harness + labeled-window methodology, produced the first trustworthy F1
(baseline 0.650 ? tuned 0.742, *fit-on-self*), per-rally failure analysis, a config sweep, and ?
via adversarial review ? found + fixed a real ingestion bug (negative-start import abort).
Cross-repo: `sports-data-collector` PR #13.
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md:8:> This iteration built the
partial-label eval harness, produced the first trustworthy
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md:19:- The video is **partially
labeled**: labels cover `0?605s` of a **23m29s** video
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md:20: (1920×1080, 59.94 fps,
84,480 frames). This made *partial-label-aware* evaluation a
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md:24:-
**`backend/eval/eval_partial_labels.py`** (+ `tests/test_eval_partial_labels.py`, 4 tests):
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md:25: scores WASB rally
segments against a human CSV **restricted to the labeled window**
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md:29:- **Methodology ? the
labeled-window restriction (the load-bearing idea):** with partial
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md:34:- **WASB trajectory for
the labeled window:** first 610s ? 36,564 frames ? WASB
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md:91: Captured as a P1
follow-up: the collector should persist `labeled_window_start/end` in
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md:121:python -m
backend.eval.eval_partial_labels \
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md:125:python -m
backend.eval.eval_partial_labels --sweep --top 14 \
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md:136:- Collector P1: persist
the labeled window + provenance/maturity in the manifest.
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md:239:python -m
backend.eval.eval_partial_labels --trajectory "%TEMP%\adarsh_avi_full_traj.csv" \
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md:257:- Code:
`backend/eval/{eval_seq_partial_labels,calibrate_local,rally_seq_proto}.py` · Checkpoints:
`../../checkpoints/2026-06-eval-baseline/`

```

#### 14. user (2026-06-10T06:12:43.572Z)

```

backend\annotations\automated_provider.py:11:step so the lineages differ (see the oracle rule
below).
backend\annotations\automated_provider.py:13:?? **Oracle rule.** The labeler MUST be a
*different model lineage* than whatever
backend\annotations\automated_provider.py:16:blind ? green while both are wrong. `lineage`
records the oracle's model family so
backend\annotations\automated_provider.py:17:callers can guard against it matching production
(`warn_if_same_lineage`).
backend\annotations\automated_provider.py:45:         "annotation_registry.get('auto',
labeler=<fn>, lineage=<model-family>')."
backend\annotations\automated_provider.py:50:def warn_if_same_lineage(oracle_lineage:
Optional[str], production_lineage: Optional[str]) -> bool:
backend\annotations\automated_provider.py:51:     """Warn (and return True) if the oracle shares
production's model lineage.
backend\annotations\automated_provider.py:53:     Encodes the oracle rule in code, not just docs:
a same-lineage oracle can't be
backend\annotations\automated_provider.py:56:     if oracle_lineage and production_lineage and
oracle_lineage == production_lineage:

```

```

backend\annotations\automated_provider.py:58:         "Oracle lineage '%s' matches
production's ? the regression test may be "
backend\annotations\automated_provider.py:59:         "blind to shared failure modes. Use a
different model lineage.",
backend\annotations\automated_provider.py:60:         oracle_lineage,
backend\annotations\automated_provider.py:67:     def __init__(self, labeler: Optional[Labeler] =
None, lineage: str = "unspecified"):
backend\annotations\automated_provider.py:72:         lineage: the oracle's model family
(e.g. 'gemini', 'qwen-vl') ? recorded
backend\annotations\automated_provider.py:73:         so callers can enforce the
oracle rule via warn_if_same_lineage.
backend\annotations\automated_provider.py:76:         self.lineage = lineage
backend\eval\regression.py:15: independent-lineage oracle (fast smoke alarm). See
docs/GOLDEN_SET_IMPLEMENTATION.md.

```

## 15. user (2026-06-10T06:13:20.776Z)

```

1      # Quality Iteration: 2026-06 Multi-video LOVO + learned-segmenter prototype
2
3      **Date:** 2026-06-08
4      **STATUS: COMPLETED** (second concrete iteration; follows `../2026-06-human-gt-tuning/`)
5
6      > The session that turned "tune one match" into "is the *method* right across matches."
Built a 2-match
7      > human-labeled corpus, ran the first honest cross-video LOVO, uncovered that the
unsupervised windowing
8      > heuristic is **footage-fragile even when WASB recall is excellent**, and **proved with
a prototype that the
9      > rally signal is learnable** where the heuristic fails. Motivates + de-risks the owned
model
10     > (`../research/OWNED_MODEL_TRAINING_STUDY.md`).
11
12     ## Corpus built this iteration
13     | video | role | rallies | res / fps | trajectory |
14     |---|---|---|---|---|
15     | Adarsh-and-Avi (full) | train | **96** | singles · 1080p / 59.94 |
`~/clips/adarsh_avi_full_traj.csv` (84,480 fr) |
16     | Kushagra-Yash-Avi | train | 32 | singles · 4K?1920 / 59.94 |
`~/clips/kushagra_traj.csv` (32,232 fr) |
17     | **LargeTestVideo** | **train** | **49** | **doubles · 5.3K?1920 / 29.97** |
`~/clips/largetestvideo_traj.csv` (23,040 fr) |
18     | **gBaaddy (20 May)** | **train** | **47** | **1080p / 59.94 · multi-court** |
`~/clips/gbaaddy_traj.csv` (52,232 fr) |
19     | **TestLargeVideo** | **train** | **6** | **5.3K?1920 / 29.97 · short** |
`~/clips/testlarge_traj.csv` (3,019 fr) |
20
21     *(Grew to **n=5 distinct matches** over the session: 2 singles + 1 doubles + 2 more,
across 3 cameras/frame-rates,
22     incl. 2 multi-court ? see the n=3 and n=5 updates below.)*
23
24     All WASB trajectories generated on one 8 GB GPU, **serialized via file-based wait-
loops** (`run_wasb*.sh`);
25     GPU never double-booked, race-free via a `kushagra_traj.csv`-exists gate). Manifest:
`output/human_lovo_manifest.json`.
26
27     ## Result 1 ? full-match Adarsh (96 rallies, IoU?0.5)
28     baseline F1 **0.632**, tuned+gate2 **0.727** (P0.80/R0.67), sweep ceiling **0.750**
(vel0.02/merge1.0/min_dur2.0/gate off).
29     vs the opening-10-min slice (40 rallies, prior iteration): 0.667 / 0.742. **Best-config
region is identical across the
30     slice and the full match** (merge_gap 1.0, min_dur 2.0; velocity irrelevant) ? tuning is
stable. Per-rally:
31     `output/adarsh_avi_full_vs_human.csv`.
32
33     ## Result 2 ? first cross-video LOVO (`calibrate_local`, the number to trust)

```



```

34     ``
35     per-video best-fit:  adarsh 0.777   |   kushagra 0.163 (ceiling ? NO config beats it)
36     leave-one-video-out: baseline mean 0.316 ? calibrated 0.443 ± 0.279 (overfit gap
+0.027, tiny)
37     held-out adarsh    0.722   (config tuned on Kushagra ? Adarsh's own optimum ? config
TRANSFERS)
38     held-out kushagra 0.163   (config tuned on Adarsh = Kushagra's own ceiling)
39     ``
40     **The asymmetry is the finding.** Tuning generalizes fine *where the heuristic can solve
the video*
41     (Kushagra?Adarsh = 0.722). But **Kushagra is unsegmentable by *any* velocity-windowing
config (0.163)** ? and
42     this is NOT a WASB-recall failure: WASB tracks the shuttle **better** on Kushagra than
Adarsh (**97% of GT
43     rallies contain ?2 net-crossings**, median 5, 310 visible pts/rally, vs Adarsh 76%). The
likely mechanism:
44     Kushagra's shuttle is visible 67.5% of the time (vs Adarsh 40%) ? tracked *through*
inter-rally activity ? the
45     "shuttle-is-moving" heuristic can't place clean boundaries (mean boundary error ~5 s).
**The ±0.279 spread is
46     method variance, not config.** This also **corrects the earlier "~0.73 baseline"** ?
that was fit-on-self; the
47     honest cross-video heuristic bar is **~0.44.**
48
49     ## Result 3 ? learned per-frame prototype (the proof) ?
`backend/eval/rally_seq_proto.py`
50     A dependency-light prototype (numpy + scipy + the repo's pure-Python
`LogisticRegression`): 6 hand-built per-frame
51     features over the WASB trajectory ? visibility rate, mean/max shuttle speed, **net-
crossing rate**, x/y spread ?
52     labeled rally(1)/non-rally(0), **honest leave-one-video-out** (train on one match, test
the other; split by match).
53
54     | held-out | per-frame AUC | learned F1 (real LOVO) | learned F1 (oracle threshold) |
heuristic ceiling |
55     |---|---|---|---|---|
56     | Adarsh | 0.878 | 0.704 | 0.721 | 0.73 |
57     | **Kushagra** | **0.841** | 0.049 | **0.578** | **0.163** |
58
59     **The signal is learnable.** Held-out per-frame AUC is **0.84?0.88** ? including on
Kushagra (trained only on
60     Adarsh), the match the heuristic *cannot* segment. With a calibrated threshold the
learned model hits
61     **Kushagra F1 0.578 ? 3.5× the heuristic's 0.163 ceiling.** The naive cross-video
threshold gives 0.049, so the
62     remaining gap is **threshold/operating-point calibration at n=2, not representation**
(the probability scale
63     shifts across footage; fixed by more videos / per-video calibration / a scale-invariant
sequence model).
64
65     > This directly answers the owned-model study's "dominant empirical unknown" ? *can a
low-dim WASB trajectory
66     > feed a learned temporal segmenter? **Yes (AUC 0.84).** The prototype is the seed of
the study's **Gen-0
67     > (ASFormer-over-WASB-features)**. ? It is a *directional* proof, not production: n=2,
linear model, hand
68     > features, threshold-transfer unsolved. The real ship gate is a segment-level A/B on
the golden set at n?5
69     > (needs the GPU/WSL machine ? not runnable in this dev container).
70
71     ## Update ? n=3: doubles match added ? **the crossover** (the final number)
72
73     Added **LargeTestVideo** (doubles, 5.3K?1920, **29.97 fps**, 49 rallies, fully tagged) ?
a different play
74     type *and* camera *and* frame rate. Single-video: heuristic best F1 **0.750** (tuned, R

```

```

0.918), best-fit 0.827
75     ? so **doubles is heuristic-friendly; Kushagra remains the lone heuristic failure.**
Per-rally:
76     `output/largetestvideo_vs_human.csv`. Manifest now n=3
(`output/human_lovo_manifest.json`).
77
78     **Honest cross-match LOVO, heuristic vs learned, n=2 ? n=3:**
79
80     | held-out match | heuristic LOVO | learned LOVO (AUC) | learned LOVO F1 |
81     |---|---|---|---|
82     | Adarsh (singles) | 0.725 | 0.878 | 0.701 |
83     | **Kushagra (singles, the hard one)** | **0.151** | 0.850 | **0.417** |
84     | LargeTest (doubles) | 0.755 | **0.914** | 0.729 |
85     | **mean held-out F1** | **0.544 ± 0.278** | ? | **0.615** |
86     | *(same metric at n=2)* | *0.443* | ? | *0.377* |
87
88     **Findings (the data-backed case, honestly):**
89     1. **The learned model OVERTAKES the heuristic at n=3** (mean **0.615 vs 0.544**); at
n=2 it *trailed*
90     (0.377 vs 0.443). The crossover the plan predicted ? the learned path **scales with
data**, the heuristic
91     does not.
92     2. **Both means rose with n, for opposite reasons.** The heuristic rose only because the
new match (doubles)
93     happens to suit it; it **still cannot rescue Kushagra at any n** (held-out 0.151,
capped). The learned model
94     rose because **more data closed its calibration gap** ? Kushagra's held-out F1 went
**0.049 ? 0.417** just
95     from one added match (oracle ceiling 0.593, AUC 0.850 ? it will keep climbing with
n).
96     3. **The decisive edge is on the *hard* footage** ? where the heuristic fails
(Kushagra), the learned model is
97     **2.8× better and rising** ; on the easy matches (Adarsh, doubles) they're comparable
(~0.72?0.75). The owned
98     model's value is exactly the generalization the heuristic lacks.
99     4. **Generalizes across play type** ? the **doubles** match, held out and trained only
on the **two singles**
100     matches, scores AUC **0.914** / F1 0.729. Different player count, camera, and frame
rate, and the per-frame
101     rally signal still transfers. (Notably *despite* the prototype's speed features not
yet being fps-normalized
102     ? 30 vs 60 fps ? which is a known refinement that should only help.)
103
104     ### Final quality metrics (n=3, honest leave-one-video-out)
105     - **Heuristic (current pipeline):** mean held-out F1 **0.544 ± 0.278** ? the ±0.278 is
the footage-fragility
106     (Kushagra ~0.15 vs others ~0.75); **hard-capped, does not improve on the videos it
can't represent.**
107     - **Learned prototype (owned-model seed):** mean held-out F1 **0.615**, per-frame **AUC
0.85?0.91**, **±0.24
108     for one added match and still climbing.** This is a *directional* number (n=3, linear
model, hand features,
109     fps-norm + per-video calibration still on the table) ? the trend, not the absolute, is
the signal.
110
111     **Conclusion (reassures the plan ? with data, not optimism):** the owned/learned path
has **already overtaken
112     the heuristic at n=3 and improves with every match**, while the heuristic is **ceilinged
by footage it cannot
113     represent**. The bottleneck is confirmed to be **data scale + calibration (n?5), not
method.** Reusing every
114     labeled video for training is the right strategy ? each one lifts the learned model and
none can fix the
115     heuristic's hard cases. ? Proceed with `../../research/OWNED_MODEL_TRAINING_STUDY.md`
and

```

```
116     `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (Gen-0 first), gated on n?5 + owner sign-off.
117
118     ## Update ? n=5 + the MULTI-COURT root cause (the defining finding)
119
120     Added **gBaaddy** (47 rallies, 1080p/60) + **TestLargeVideo** (6 rallies, 5.3K/30 ? **too
short to be a reliable
```

## 16. user (2026-06-10T06:13:22.073Z)

```
75     5. **Fireworks** (managed, contractual no-train + ZDR) usable; **Together BLOCKED**
until a signed DPA + explicit no-train clause. AWS SageMaker / Azure ML = heavy for a solo
founder; India-residency only.
76
77     ## Label reuse ? one corpus, three transcoders (the actual moat)
78
79     Promote the rally CSVs from eval-only files to **one model-agnostic corpus**: JSONL, one
row per
80     `(video_id, rally_ordinal)` = `{start, end, ending_reason, sport, source
(human|pseudo|vendor),
81     annotator/provenance, consent/opt-in, label_type, split (BY MATCH not clip),
trajectory_ref}`. The
82     provenance/consent fields are the **enforcement point** for the legal guardrails. Three
deterministic
83     transcoders read it, zero re-labeling:
84     1. **Heuristic/LogReg view** ? already exists
(`backend/eval/training.py::label_candidates` via `metrics.match_intervals`).
85     2. **TAL view** ? ActivityNet JSON (`segment:[start,end]`, `label:'rally'`); our seconds
map 1:1.
86     3. **VLM clip?QA view** ? "list every rally [start,end] in this clip"; `ending_reason` ?
a second QA task.
87
88     **One scorer spine:** reuse `metrics.match_intervals` for heuristic-labeling, TAL eval,
AND the Gen-2 RFT
89     reward ? no metric drift. **Leakage guard:** split **by match, never by clip**.
**Amplification (all
90     guardrail-clean):** open-teacher pseudo-labels (WASB/own-model, confidence-gated ? never
paid-LLM),
91     boundary-preserving temporal augmentation, active learning to spend the Roora budget on
highest-uncertainty
92     matches. ? **The new partial-label windowing**
(`eval_partial_labels.restrict_to_window`) must flow into the
93     TAL/VLM views so prefix-labeled videos aren't mis-scored (a corpus that's partially
tagged is the norm).
94
95     ## Licensing & privacy guardrails (all respected; verify per-checkpoint)
96
97     - **Bases: Apache-2.0/MIT only.** APPROVED: Qwen2.5-VL-7B/32B (Apache-2.0),
Qwen3-VL-4B/8B (Apache-2.0),
98     Gemma 4 E2B/E4B/12B-Unified (Apache-2.0 + native video/audio), InternVL3 ?38B
(MIT/Apache), ASFormer/
99     ActionFormer/TriDet (MIT). **REJECTED:** Gemma 3 (viral Gemma Terms + PUP),
Qwen2.5-VL-3B (NC research),
100     Qwen-72B/InternVL3-78B (100M-MAU cap), Llama Vision (700M-MAU), DAMO VideoLLaMA3
weights (NC + paid-LLM-trained),
101     TimeChat/VTimeLLM/Momentor (NC/no-license = method-reference only), VideoMAE weights
(CC-BY-NC), `yabufarha/ms-tcn`
102     (Commons Clause ? use `sj-li/MS-TCN2` MIT). ? **Gemma 4 size names =
E2B/E4B/26B-A4B/31B/12B-Unified ? there is no plain "4B".**
103     - **No paid-LLM training data** ? Gemini/OpenAI/Anthropic = inference/eval/teacher-
reference only, **never** a
104     training signal. Gemini silver labels stay out of every transcoder.
105     - **Exclude minors by default + separate training opt-in;** per-user adapters trained
only on that user's
106     consented video (deletion = drop the adapter). Enforced at the corpus-membership layer
before transcoding.
```

```

107 - **Privacy lever = local inference** (Gen-0 ~1.1M params; Gen-2 4B quantized). Any
cloud *training* needs a
108 signed DPA + no-train assurance; route only owned/consented footage, never user
uploads.
109
110 ## Cost (spend on labels, not GPUs)
111
112 - **Gen-0:** ~$0 (local 2070 Super; trains in minutes; ~4.5 GB VRAM).
113 - **Gen-2 LoRA:** ~$240/run on a rented 24?48 GB GPU (RunPod cheapest; **VRAM/token
cost on real 1080p/60fps
114 clips is UNVERIFIED ? 24 GB is a floor, re-benchmark**). Managed (Fireworks/Together)
~$2?30 *if* text-priced

```

## 17. assistant (2026-06-10T06:13:51.194Z)

One last check ? distill\_local's label source and the gitignore status of output manifest.

## 18. user (2026-06-10T06:13:52.272Z)

```

1  """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3  Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4  further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5  using a few resolution/fps-INVARIANT local features, so the model transfers across
6  videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8  Flow (all local at runtime):
9      WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10      filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11      active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12      -> rally windows.
13
14  Training labels come from Gemini full-context labels via the SAME IoU matching the
15  evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16  other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17  compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18  rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20  Run:
21      python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22  """
23  from __future__ import annotations
24
25  import json
26  import argparse
27  from typing import Dict, List, Tuple
28
29  import numpy as np
30
31  from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
32  from backend.pipeline.detectors import rally_gate
33  from backend.eval.calibrate_wasb import load_golden_gts
34  from backend.eval.training import label_candidates
35  from backend.eval.classifier import LogisticRegression
36  from backend.eval.metrics import score
37  from backend.eval.gemini_refine import merge_overlaps
38
39  Interval = Tuple[float, float]
40
41  FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
42  # Recall-oriented proposal: deliberately permissive so real rallies are among the
43  # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
44  PROPOSAL = (0.005, 1.0, 0.5)

```

```
45     BASELINE_LOCAL = (0.02, 2.0, 2.0)    # today's production-ish windowing, no learning
46
47
48     def build_candidates(trajecory_csv: str, fps: float, frame_width: float,
49                          proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
50         """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
```

**19. user (2026-06-10T06:14:22.497Z)**

Structured output provided successfully

**20. assistant (2026-06-10T06:14:26.487Z)**

Verification complete.